

AMAR TELIDJI UNIVERSITY

Laghouat

Computer Sciences Department

Class: **4th Year Cybersecurity**

Teacher: **Dr. Leila BENAROUS**

Student: **Smail Mersad**

COURSE

Wireless networks security

REPORT TITLE

Centralized AAA with RADIUS

2025 / 2026

Introduction

This lab focuses on analyzing security weaknesses in wireless networks, Android applications, and Linux systems. The exercises cover Wi-Fi traffic analysis, WPA/WPA2 handshake attacks, rogue access points, Android reverse engineering, mobile traffic interception, Android manifest vulnerabilities, Linux file protection, and Linux capabilities.

The goal is to extract technical information from captures, applications, and system configurations, then explain the related vulnerability, possible exploitation, and suitable mitigation.

Objectives

The objectives of this lab are to:

- Extract Wi-Fi and Bluetooth traffic information using tools such as Wireshark, [airodump-ng](#), and [tshark](#).
- Analyze WPA/WPA2 handshakes and understand dictionary attack risks.
- Detect rogue access points and understand Evil Twin attack scenarios.
- Reverse engineer Android APKs to identify hardcoded data and insecure storage.
- Test mobile application traffic interception using Mitmproxy.
- Analyze Android permissions and exported components using [apktool](#).
- Apply Linux hardening techniques using [chattr](#), [getcap](#), and [setcap](#).
- Propose mitigations based on each identified vulnerability.

Exercise 01:

```
Cybersecurity [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
kali@kali: ~
Session Actions Edit View Help

CH 0 ][ Elapsed: 36 s ][ 2026-04-07 10:07 ][ Finished reading input file wpa2-psk-linksys.cap.

BSSID      PWR Beacons  #Data, #/s  CH  MB  ENC CIPHER AUTH ESSID
00:0B:86:C2:A4:85  0      85      44  0  1  11  . WPA2 CCMP PSK linksys

BSSID      STATION    PWR   Rate Lost  Frames Notes Probes
00:0B:86:C2:A4:85  00:13:CE:55:98:EF  0    0 - 0    0    234 PMKID linksys
```

The screenshot shows the [airodump-ng](#) output for the capture file [wpa2-psk-linksys.cap](#). It displays the detected access point, its BSSID, channel, encryption method, authentication mode, and SSID.

Extracted Information

Field	Extracted Value	Explanation
Capture file	wpa2-psk-linksys.cap	File analyzed with airodump-ng .
SSID	linksys	Wireless network name advertised by the AP.
BSSID	00:0B:86:C2:A4:85	MAC address identifying the access point.
Channel	1	Wi-Fi channel used by the AP.
Security	WPA2 / CCMP / PSK	WPA2-Personal using CCMP encryption and a pre-shared key.
Frame type	Beacon frames	Management frames used by the AP to announce the network.

Analysis and Vulnerability

The beacon frames reveal the basic configuration of the wireless network, including the SSID, BSSID, channel, and security mode. This information is broadcast in plaintext because beacon frames are management frames required for normal Wi-Fi operation.

The vulnerability is **wireless reconnaissance through exposed management-frame information**. Even though WPA2 protects data traffic, an attacker nearby can still collect metadata about the network without connecting to it.

Exploitation

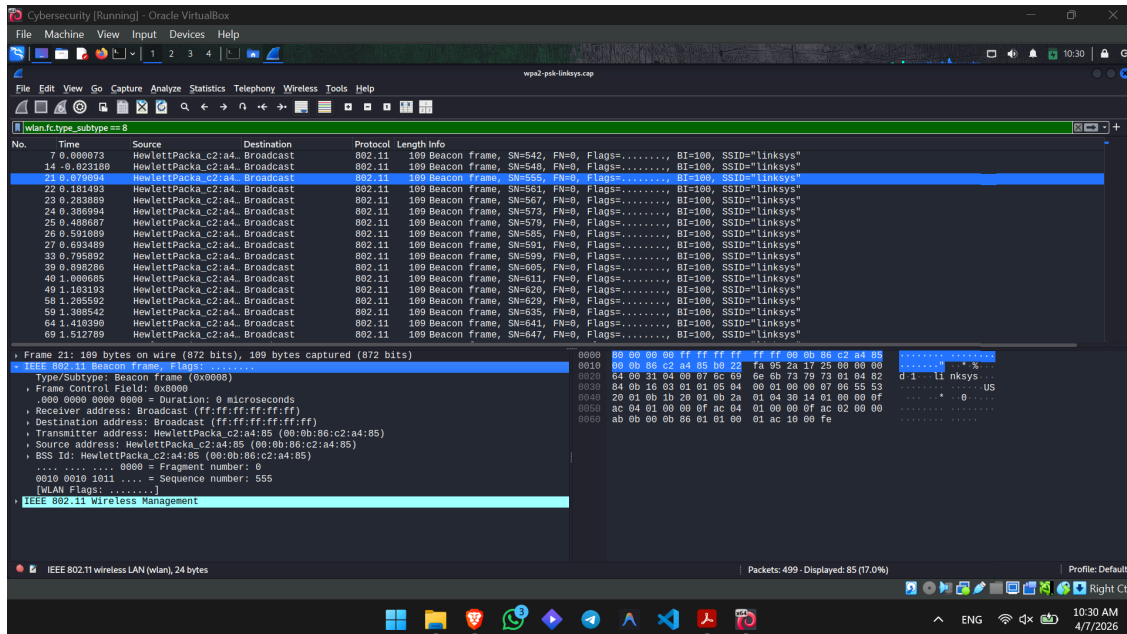
An attacker can use the SSID and BSSID to identify the target AP, then tune their wireless adapter to channel 1 to monitor the network. Since the network uses WPA2-PSK, the attacker may try to capture a 4-way handshake and perform an offline dictionary attack if the passphrase is weak.

Protection / Mitigation

Use a long and random WPA2/WPA3 passphrase, disable WPS, and monitor abnormal wireless activity such as deauthentication frames or rogue APs using tools like Kismet. Where possible, use WPA3 because it offers stronger protection against password-guessing attacks.

Exercise 02:

2.1 Beacon Frames — `wlan.fc.type_subtype == 8`



The screenshot shows Wireshark filtering beacon frames from the Wi-Fi capture. Beacon frames are broadcast by the access point to announce the wireless network. Your basic report identifies this filter as the one used to display router/AP beacon frames.

Extracted Information

Field	Extracted Value	Explanation
Capture file	wpa2-psk-linksys.cap	Wi-Fi capture analyzed in Wireshark.
Display filter	wlan.fc.type_subtype == 8	Filters beacon management frames.
Frame type	Beacon frame	Sent periodically by the AP to announce the network.
Security meaning	Network discovery	Reveals SSID, BSSID, channel, and capabilities.

Analysis and Vulnerability

Beacon frames are necessary for Wi-Fi operation, but they expose network metadata in plaintext. An attacker can use them to discover nearby networks and identify possible targets.

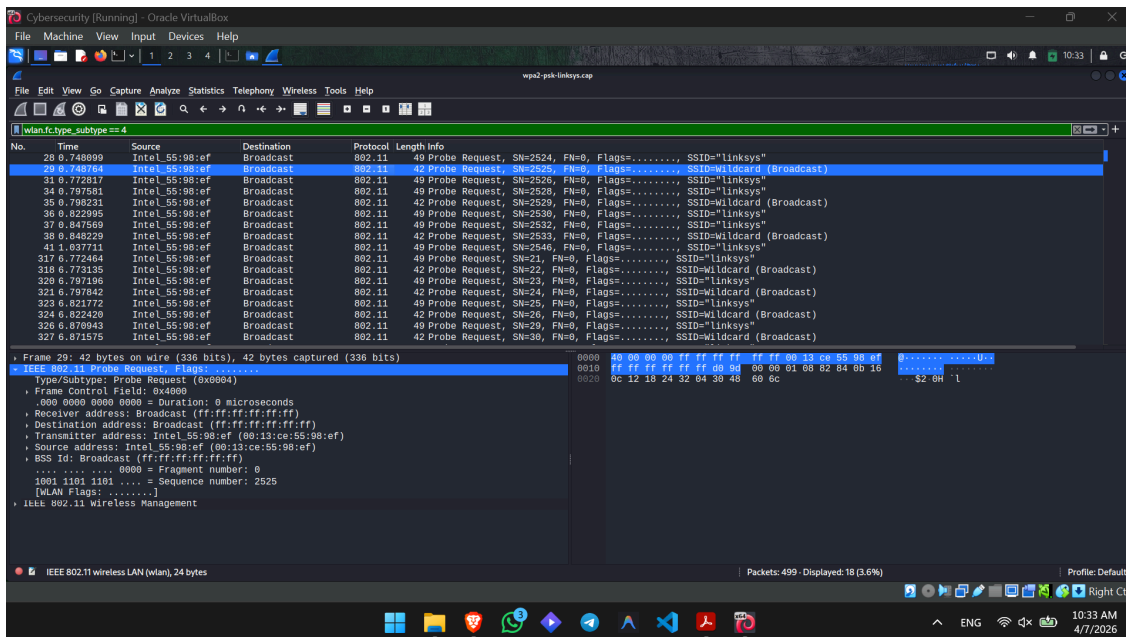
Exploitation

An attacker can generate many fake beacon frames with random SSIDs, creating beacon flooding. This can confuse users and make the Wi-Fi network list unreliable.

Protection / Mitigation

Use wireless monitoring tools to detect abnormal beacon activity, avoid connecting to unknown SSIDs, and verify trusted networks before joining.

2.2 Probe Request Frames — `wlan.fc.type_subtype == 4`



The screenshot shows client devices sending probe request frames. These frames are used when a device searches for available or previously known networks.

Extracted Information

Field	Extracted Value	Explanation
Display filter	<u>wlan.fc.type_subtype == 4</u>	Filters probe request frames.
Frame type	Probe Request	Sent by clients looking for Wi-Fi networks.
Sender	Wireless client	The device searching for a network.
Security meaning	Client tracking risk	May reveal networks the device has connected to before.

Analysis and Vulnerability

Probe requests can leak information about a user's preferred networks. This allows nearby attackers to profile or track devices based on the SSIDs they search for.

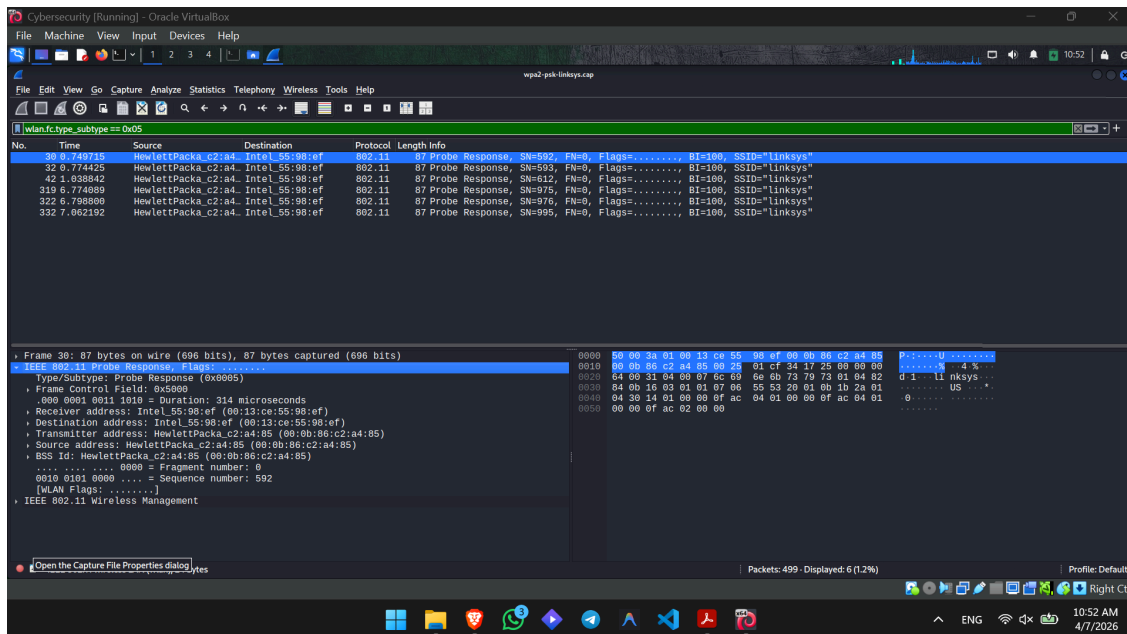
Exploitation

An attacker can passively collect probe requests and build a profile of the user's movements, such as places they have visited, worked, or lived.

Protection / Mitigation

Disable auto-join for unnecessary networks, forget old Wi-Fi profiles, and use devices that randomize MAC addresses during active scanning.

2.3 Probe Response Frames — wlan.fc.type_subtype == 0x05



The screenshot shows probe response frames, where an AP replies to a client’s probe request. Your report links this frame type to rogue AP and Evil Twin attacks.

Extracted Information

Field	Extracted Value	Explanation
Display filter	<u>wlan.fc.type_subtype == 0x05</u>	Filters probe response frames.
Frame type	Probe Response	AP response to a client searching for a network.
Sender	Access point	The AP replies with network details.
Security meaning	Rogue AP risk	Attackers can forge responses to attract clients.

Analysis and Vulnerability

Probe responses can be abused because clients may trust the response and attempt to connect. This is dangerous when attackers impersonate a known network.

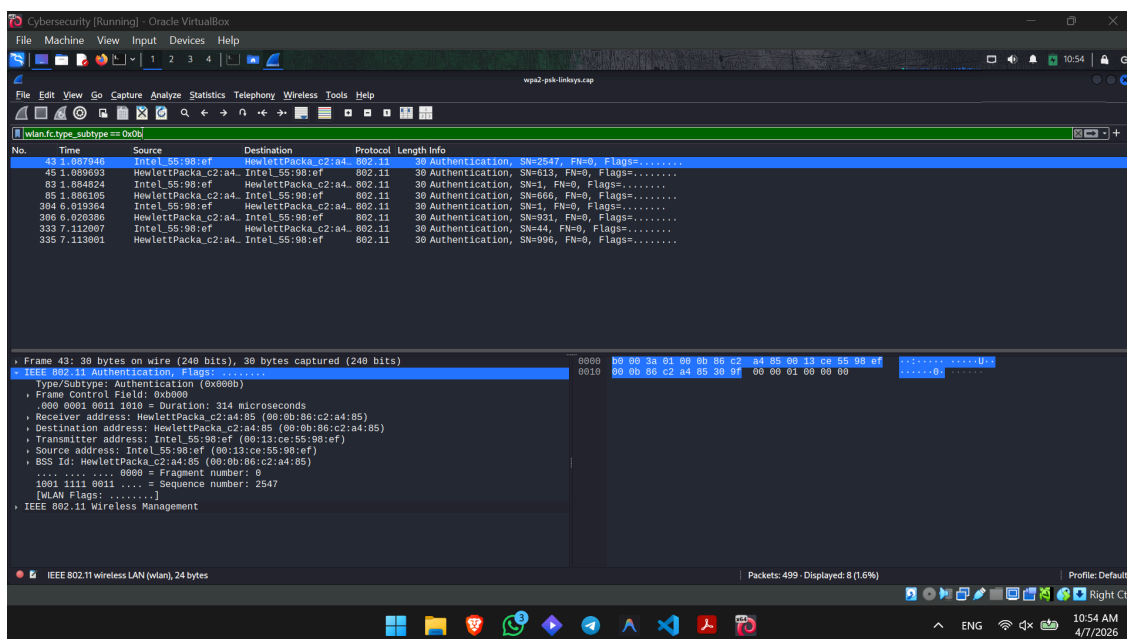
Exploitation

An attacker can create a rogue AP or Evil Twin that responds to a client's probe request. The client may connect to the fake AP, allowing interception or further attacks.

Protection / Mitigation

Avoid auto-connecting to open networks, verify network identity, use VPN on untrusted Wi-Fi, and deploy wireless intrusion detection to detect Evil Twin APs.

2.4 Authentication Frames — `wlan.fc.type_subtype == 0x0b`



The screenshot shows authentication frames, which appear during the initial connection process between a client and an AP.

Extracted Information

Field	Extracted Value	Explanation
Display filter	wlan.fc.type_subtype == 0x0b	Filters authentication frames.
Frame type	Authentication	Initial step where a client attempts to authenticate with the AP.
Protocol	IEEE 802.11 management	Used before association and encrypted communication.
Security meaning	Connection-state exposure	Shows connection attempts and can be abused for flooding.

Analysis and Vulnerability

Authentication frames are part of normal Wi-Fi connection setup, but an AP has limited resources to track connection attempts. This creates a denial-of-service risk.

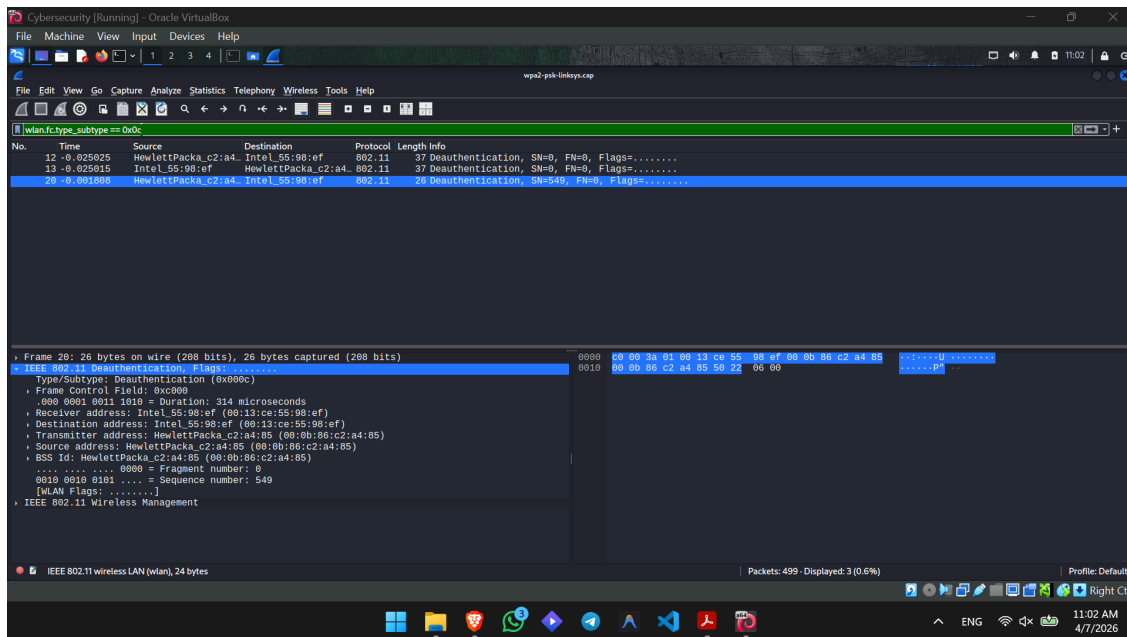
Exploitation

An attacker can spoof many MAC addresses and send large numbers of authentication requests, filling the AP's client table and preventing legitimate users from connecting.

Protection / Mitigation

Use APs with management-frame protection and DoS detection, rate-limit authentication attempts where supported, and monitor for unusual numbers of authentication frames.

2.5 Deauthentication Frames — wlan.fc.type_subtype == 0x0c



The screenshot shows deauthentication frames. These frames force a client to disconnect from the network and are often abused to cause denial of service or force a new WPA handshake.

Extracted Information

Field	Extracted Value	Explanation
Display filter	<u>wlan.fc.type_subtype == 0x0c</u>	Filters deauthentication frames.
Frame type	Deauthentication	Disconnects a client from the AP.
Security meaning	DoS and handshake capture risk	Can force clients offline and trigger reconnection.
Common abuse	Deauth attack	Used to interrupt users or capture a fresh handshake.

Analysis and Vulnerability

Deauthentication frames are dangerous because, in many networks, management frames are not protected. Attackers can forge them and force clients to disconnect.

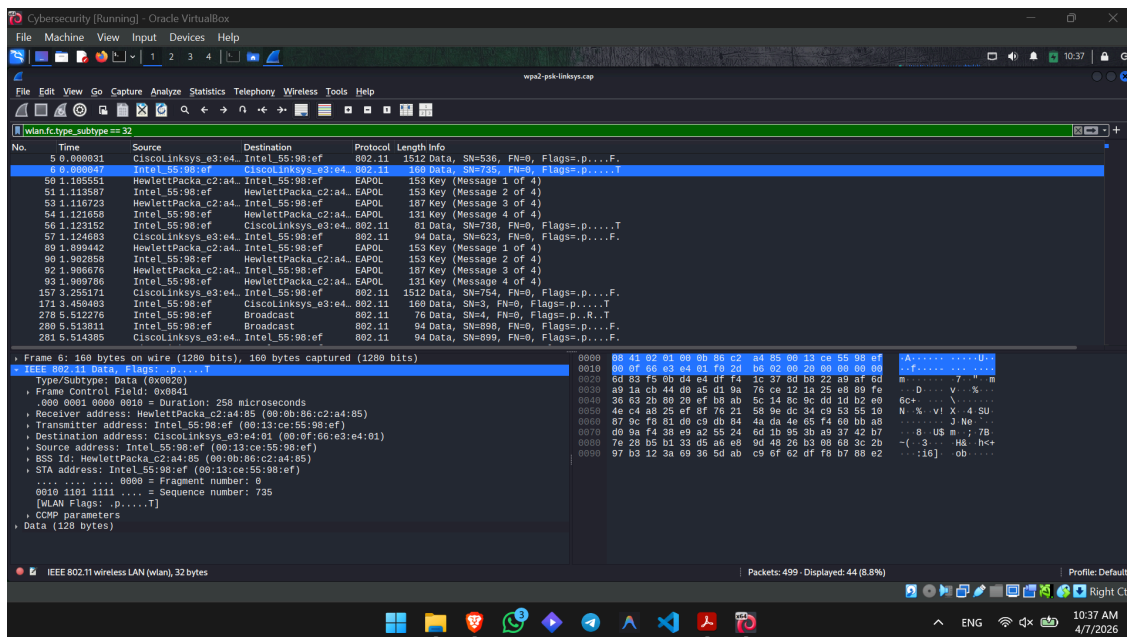
Exploitation

An attacker can repeatedly send forged deauthentication frames to create a Wi-Fi denial of service. They can also force a client to reconnect, allowing them to capture the 4-way WPA handshake for offline password guessing.

Protection / Mitigation

Enable Protected Management Frames where supported, use WPA3 when possible, monitor for deauthentication floods, and use strong WPA2/WPA3 passphrases.

2.6 Encrypted Data Frames — `wlan.fc.type_subtype == 32`



The screenshot shows encrypted 802.11 data traffic after the client has connected to the access point. Your report identifies this filter as showing actual encrypted data passing between devices.

Extracted Information

Field	Extracted Value	Explanation
Display filter	<code>wlan.fc.type_subtype == 32</code>	Filters encrypted data frames.

Frame type	Data frame	Carries user traffic after connection.
Protection	Encrypted	Data is protected by WPA2 encryption.
Security meaning	Confidentiality depends on key strength	Weak PSK can still expose traffic if cracked.

Analysis and Vulnerability

Encrypted data frames protect user traffic, but the protection depends on the strength of the WPA2 passphrase. If the PSK is weak and the handshake is captured, the attacker may decrypt traffic later.

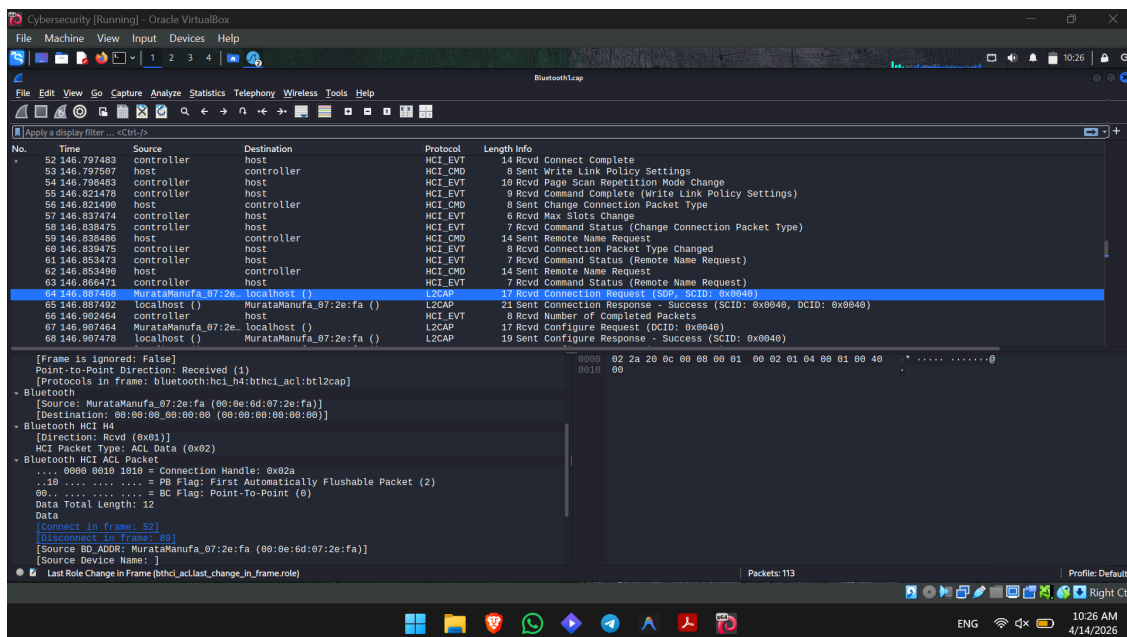
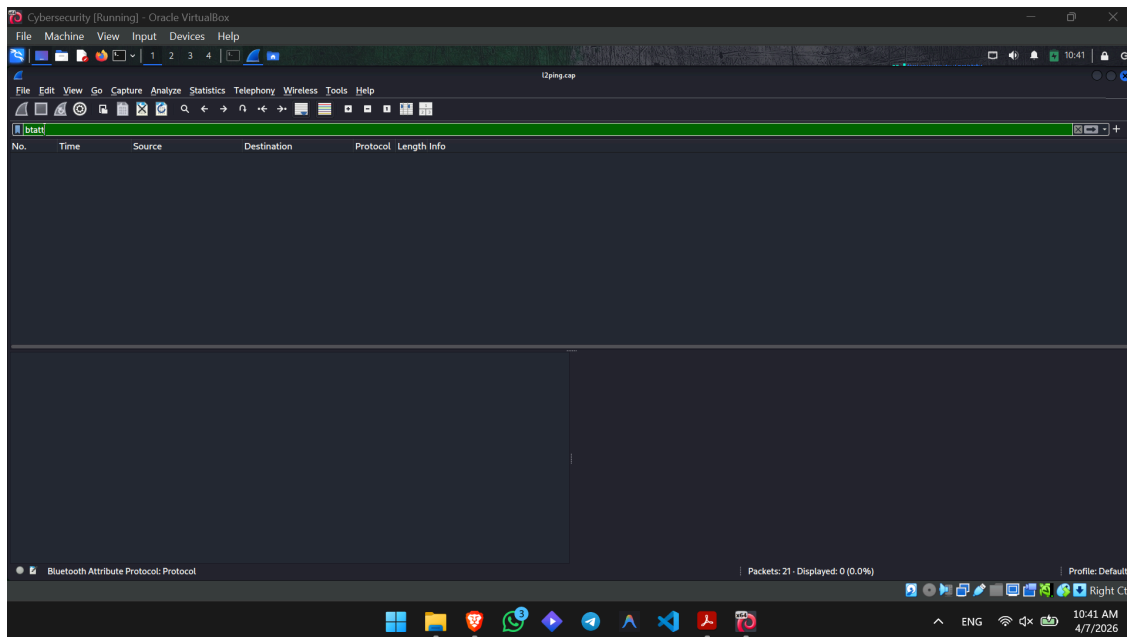
Exploitation

An attacker who captures the handshake and cracks the password can decrypt captured data frames and potentially inspect network traffic.

Protection / Mitigation

Use a strong random WPA2/WPA3 password, rotate weak passwords, and avoid shared predictable passphrases.

2.7 Bluetooth Traffic — [btatt](#)



The screenshot shows the use of the [btatt](#) filter on the Bluetooth capture. The filter returned no ATT packets because the capture contains classic Bluetooth L2CAP traffic, not BLE Attribute Protocol traffic.

Extracted Information

Field	Extracted Value	Explanation
Capture files	l2ping.cap , Bluetooth1.cap	Bluetooth captures analyzed in Wireshark.

Display filter	btatt	Filters Bluetooth ATT packets.
Result	No ATT packets displayed	The capture contains classic Bluetooth traffic, not BLE ATT.
Protocols observed	HCI, L2CAP, SDP	Classic Bluetooth protocols found in the capture.
Target MAC address	00:0e:6d:07:2e:fa	Bluetooth device identity found in the capture.
Pairing type	Legacy PIN pairing	Older pairing method with weaker security.
Link key	ec596f3306bba9e53d7b72de47c1404a	Plaintext 128-bit combination key extracted from packet 21.

Analysis and Vulnerability

The [btatt](#) filter was not suitable for this capture because ATT is mainly used in Bluetooth Low Energy, while this file contains classic Bluetooth L2CAP traffic. However, the capture still reveals sensitive information, including device identity, legacy pairing, and a plaintext link key. The report identifies legacy PIN pairing and cryptographic exposure in HCI as the main weaknesses.

Exploitation

An attacker may capture legacy pairing traffic and attempt offline PIN cracking. If the plaintext link key is extracted from the host system, the attacker can spoof the trusted device and connect without repeating the pairing process.

Protection / Mitigation

Avoid legacy Bluetooth pairing, use Secure Simple Pairing or newer Bluetooth security modes, remove old bonded devices, and protect local Bluetooth key storage from unauthorized access.

Exercise 03:

```
(kali@kali)-[~]
$ tshark -r wpa2-psk-linksyes.cap -Y "eapol" -T fields -e frame.number -e wlan_rsn_eapol.keydes.nonce -e wlan_rsn_eapol.keydes.mic -e wlan_rsn_eapol.key
es.key_info
50 ae12a150652e9bc22063720c5081e9eb74077fb19fffe871dc4ca1e6f448af85 00000000000000000000000000000000 0x008a
51 e8dfa16b8769957d8249a4ec68d2b7641d3782162ef0dc37b014cc48343e8dd2 56f98b98da5d55e3be396b43c7eb012a 0x010a
53 ae12a150652e9bc22063720c5081e9eb74077fb19fffe871dc4ca1e6f448af85 66ae84a96f7c83c2f4717e9d4c2285c7 0x13ca
54 00000000000000000000000000000000000000000000000000000000000000 41e261886db4de641122c7c224026051 0x030a
89 87c3b0fb38effd2c224d5f670e3c58ace8a3028fc0f6e4e4dc6f6ec18ef91cf8 00000000000000000000000000000000000000000000 0x008a
90 e8dfa16b8769957d8249a4ec68d2b7641d3782162ef0dc37b014cc48343e8dd2 8d2e59b89c1570584a0ebf011a597f29 0x030a
92 87c3b0fb38effd2c224d5f670e3c58ace8a3028fc0f6e4e4dc6f6ec18ef91cf8 7c6e612dce56c1e8cc9cf3026d755e46 0x13ca
93 00000000000000000000000000000000000000000000000000000000000000 0efd5bd62149cb4349623b08795f7aed 0x030a
339 1a9bdf0cc89e5e3220f71aa74fe32df65bb8c1c5b8664b9d98aef709b9644d29 00000000000000000000000000000000000000000000 0x008a
340 e8dfa16b8769957d8249a4ec68d2b7641d3782162ef0dc37b014cc48343e8dd2 0e71a625faade7ce9c8221f7b1dbce46 0x010a
343 1a9bdf0cc89e5e3220f71aa74fe32df65bb8c1c5b8664b9d98aef709b9644d29 d497d0f3a5ce0b82deb06413345e9233 0x13ca
344 00000000000000000000000000000000000000000000000000000000000000 96929b9b1280a1b78fcd06788846f008 0x030a
```

The screenshots show the decoded EAPOL 4-way handshake from the capture file [wpa2-psk-linksyes.cap](#). The exercise focuses on identifying the cryptographic fields exchanged during the handshake, especially the ANonce, SNonce, MIC, and key information values. The lab sheet specifically asks to use [tshark](#) to decode EAPOL handshake messages and extract these fields.

Extracted Information

Frame / Step	Direction	Key Info	Nonce	MIC
Frame 50 / Message 1	AP → Client	0x008a	ANonce: ae12a150652e9bc22063720c5081e9eb74077fb19fffe871dc4ca1e6f448af85	00000000000000000000000000000000 0000000000
Frame 51 / Message 2	Client → AP	0x010a	SNonce: e8dfa16b8769957d8249a4ec68d2b7641d3782162ef0dc37b014cc48343e8dd2	56f98b98da5d55e3be396b43c7eb012a
Frame 53 / Message 3	AP → Client	0x13ca	ANonce: ae12a150652e9bc22063720c5081e9eb74077fb19fffe871dc4ca1e6f448af85	66ae84a96f7c83c2f4717e9d4c2285c7
Frame 54 / Message 4	Client → AP	0x030a	SNonce: 00 0000000000000000000000000000000000	41e261886db4de641122c7c224026051

Explanation

[Message 1](#) is sent from the access point to the client. It contains the AP's random number, called the **ANonce**. The MIC is empty at this stage because the key confirmation process has not been completed yet.

[Message 2](#) is sent from the client to the access point. It contains the client's random number, called the **SNonce**, and a MIC. This MIC proves that the client can calculate the correct key from the WPA2 pre-shared password.

[Message 3](#) is sent from the access point to the client. It repeats the ANonce and includes the AP's MIC. This confirms that the AP has also derived the correct encryption keys. The key info value shows that the install bit is set, meaning the client should prepare to install the encryption keys.

[Message 4](#) is the final acknowledgment from the client to the access point. The SNonce field is zeroed because it was already sent in Message 2. The secure bit is set, and after this message the handshake is complete.

Analysis and Vulnerability

The EAPOL 4-way handshake allows the client and AP to prove they both know the WPA2 pre-shared key without sending the actual password over the air. However, the handshake exposes enough cryptographic material for offline password testing: the ANonce, SNonce, MIC, and MAC addresses.

The vulnerability is **offline dictionary attack risk against WPA2-PSK**. If the Wi-Fi password is weak, an attacker can use the captured handshake to test many password guesses without interacting with the network again.

Exploitation

An attacker can capture the handshake by passively waiting for a client to connect or by forcing a reconnection using deauthentication frames. Once the attacker captures the handshake, especially Messages 1 and 2, they can test password guesses offline. If a guessed password generates the same MIC as the captured one, the attacker has discovered the correct Wi-Fi passphrase.

Protection / Mitigation

Use a long, random WPA2/WPA3 passphrase that is not based on dictionary words, names, dates, or phone numbers. Disable WPS, monitor for repeated deauthentication frames, and use WPA3-SAE where possible because it gives stronger protection against offline password-guessing attacks.

Exercise 04:

```
(kali@kali) [~/Desktop/Lab bundle-20260420]
$ aircrack-ng -w /usr/share/wordlists/rockyou.txt wpa-psk-linksyes.cap
Reading packets, please wait...
Opening wpa-psk-linksyes.cap
Read 587 packets.

# BSSID      ESSID      Encryption
1 00:0B:86:C2:A4:85  linksys    WPA (1 handshake)

Choosing first network as target.

Reading packets, please wait...
Opening wpa-psk-linksyes.cap
Read 587 packets.

1 potential targets

Aircrack-ng 1.7

[00:00:01] 7943/10303727 keys tested (5293.68 k/s)
Time left: 32 minutes, 24 seconds                                0.08%

KEY FOUND! [ dictionary ]

Master Key   : 1D CC 47 DC 89 68 48 0B CD BB 42 09 68 33 FA 88
              69 A3 6E 17 B5 8C 1D BC 71 78 B9 D9 EF BB 89 1C

Transient Key : A3 A3 A3 A3 E7 0D 51 A9 F4 D4 70 FD DC 54 1E 84
              DF A0 4E 88 5E 36 BE E2 36 06 DC F1 12 D3 23 C5
              BE 56 FF 7F 80 42 D3 EB 1D A2 D8 F7 46 3C 31 9D
              E4 9C 26 9A 5D 13 63 12 F2 AF DC 7D 45 65 F8 91

EAPOL HMAC   : CF BB 6C 71 D5 0F 76 57 66 A1 CE 21 42 3E 9D E9

(kali@kali) [~/Desktop/Lab bundle-20260420]
$ aircrack-ng -w /usr/share/wordlists/rockyou.txt wpa.cap
Reading packets, please wait...
Opening wpa.cap
Read 13 packets.

# BSSID      ESSID      Encryption
1 00:0D:93:EB:B0:8C  test       WPA (1 handshake)

Choosing first network as target.

Reading packets, please wait...
Opening wpa.cap
Read 13 packets.

1 potential targets

Aircrack-ng 1.7

[00:00:34] 131881/14344392 keys tested (3839.95 k/s)
Time left: 1 hour, 1 minute, 41 seconds                          0.92%

KEY FOUND! [ biscotte ]

Master Key   : F9 82 31 4B 9B A4 9A 93 68 60 9C 61 CF 03 76 24
              2D 2C 17 CF E8 B0 10 6D D5 E4 90 45 29 10 06 C3

Transient Key : 45 99 63 BE 75 D5 03 A9 60 41 8C 02 04 85 FB DA
              92 7E AD 2C 39 E7 20 2B 1D 47 AA 10 4C 4E AD F3
              E1 80 CE 85 9B AF 2E A8 62 8D F0 2D C5 C1 51 CD
              FE 12 D8 95 F8 58 F5 9C E7 B7 CE FD 0F A0 FD 62

EAPOL HMAC   : 68 59 78 EE 3A F2 E0 B8 ED D9 67 8F E8 29 D7 3D
```

Extracted Information

Capture File	BSSID	ESSID	Encryption	Packets Read	Wordlist Used	Recovered Key
wpa.cap	00:0D:93:EB:B0:8C	test	WPA — 1 handshake	13	rockyou.txt	biscotte
wpa-psk-linksyes.cap	00:0B:86:C2:A4:85	linksys	WPA — 1 handshake	587	rockyou.txt	dictionary

Explanation

In both captures, Aircrack-ng detected a valid WPA handshake. This handshake contains enough cryptographic information to test password guesses offline. The real password is not displayed directly in the capture, but Aircrack-ng compares each password from the wordlist with the handshake until it finds a matching result.

For [wpa.cap](#), the key was found after testing [131,881](#) passwords, and the recovered passphrase was [biscotte](#). For [wpa-psk-linksyes.cap](#), the key was found after testing [7,943](#) passwords, and the recovered passphrase was [dictionary](#).

Analysis and Vulnerability

The vulnerability demonstrated in this exercise is **weak WPA/WPA2-PSK passphrases vulnerable to offline dictionary attacks**.

WPA does not transmit the actual Wi-Fi password in plaintext. However, if an attacker captures the 4-way handshake, they can test password guesses offline using a tool such as Aircrack-ng. If the password is short, common, or present in a public wordlist like [rockyou.txt](#), it can be recovered quickly.

The recovered keys [biscotte](#) and [dictionary](#) are weak because they are dictionary-based passwords. This proves that the strength of WPA/WPA2-PSK depends mainly on the complexity of the chosen passphrase.

Exploitation

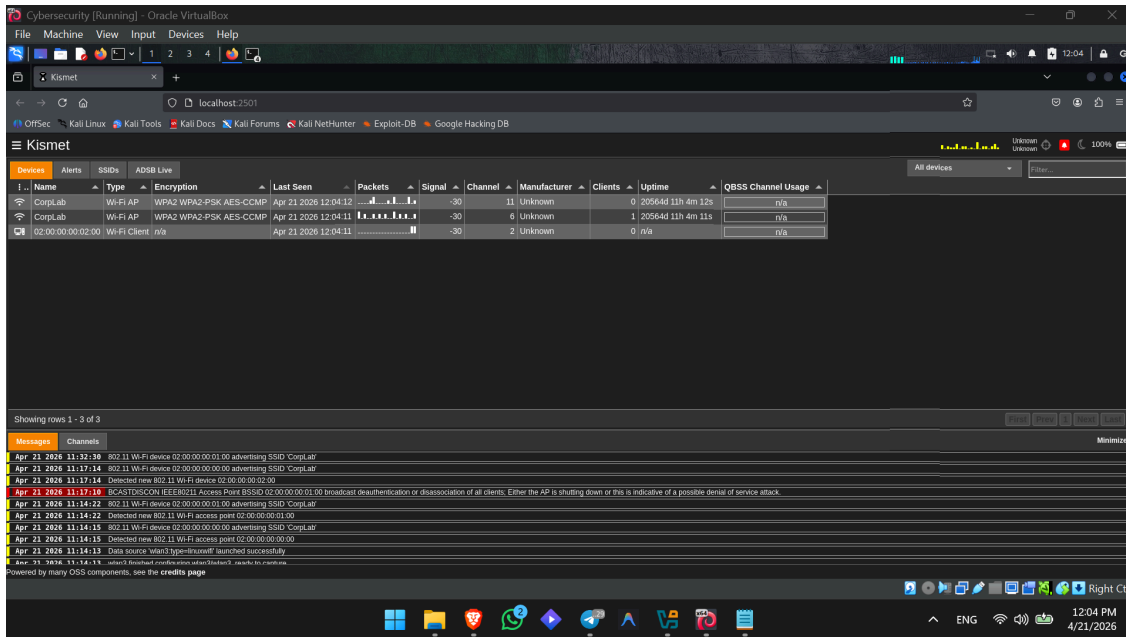
An attacker can monitor a Wi-Fi network and capture the WPA handshake when a client connects. If needed, the attacker may force a client to reconnect using deauthentication frames, causing a new handshake to appear. After capturing the handshake, the attacker can perform the attack offline without staying connected to the target network.

In this exercise, the provided capture files already contained the handshakes, so Aircrack-ng was able to use [rockyou.txt](#) to recover the passwords.

Protection / Mitigation

Use long and random WPA2/WPA3 passphrases that are not based on dictionary words, names, dates, or common patterns. Avoid passwords like [dictionary](#), [12345678](#), or simple words. Disable WPS, monitor for suspicious deauthentication attacks, and use WPA3-SAE where possible because it provides stronger protection against offline password guessing.

Exercise 05:



```
(kali@kali)-[~]
$ sudo hostapd /tmp/hostapd-legit.conf
[sudo] password for kali:
wlan0: interface state UNINITIALIZED->ENABLED
wlan0: AP-ENABLED
wlan0: STA 02:00:00:00:02:00 IEEE 802.11: authenticated
wlan0: STA 02:00:00:00:02:00 IEEE 802.11: associated (aid 1)
wlan0: AP-STA-CONNECTED 02:00:00:00:02:00
wlan0: STA 02:00:00:00:02:00 RADIUS: starting accounting session E78E130AAA9CAB61
wlan0: STA 02:00:00:00:02:00 WPA: pairwise key handshake completed (RSN)
wlan0: EAPOL-4WAY-HS-COMPLETED 02:00:00:00:02:00
```

The screenshots show the rogue AP defense lab using Kismet. In this exercise, [mac80211_hwsim](#) was used in Kali to simulate a legitimate AP, a rogue AP, a client, and a monitor interface. The legitimate AP and rogue AP both advertised the same SSID, [CorpLab](#), but they used different BSSIDs. Kismet was then used to monitor the wireless environment and detect both access points.

Extracted Information

Component	Interface	SSID	BSSID	Role / Explanation
Legitimate AP	wlan0	CorpLab	02:00:00:00:00:00	Trusted access point created in the isolated lab.
Rogue AP	wlan1	CorpLab	02:00:00:00:01:00	Fake AP using the same SSID as the legitimate network.

Client	wlan2	CorpLab	Connected to 02:00:00:00:00:00	Client configured to connect only to the legitimate AP.
Monitor interface	wlan3	N/A	N/A	Used by Kismet to detect both APs.
Protection method	wpa_supplicant with bssid= option	N/A	Legitimate BSSID only	Prevents the client from connecting to the rogue AP.

Explanation of the Extracted Data

The lab created two access points with the same SSID, [CorpLab](#). This simulates a rogue AP or Evil Twin scenario, where an attacker creates a fake access point using the same network name as the legitimate one.

The important difference between the two APs is the BSSID. The legitimate AP used [02:00:00:00:00:00](#), while the rogue AP used [02:00:00:00:01:00](#). Kismet detected both APs, allowing the administrator to identify that two different devices were advertising the same SSID. The client was configured with the [bssid=](#) option in [wpa_supplicant](#), forcing it to associate only with the legitimate AP. The verification showed that the client connected to [02:00:00:00:00:00](#), proving that access to the rogue AP was prevented.

Analysis and Vulnerability

The vulnerability demonstrated here is the **rogue access point / Evil Twin attack**. Wireless clients often trust networks based mainly on the SSID. If a rogue AP uses the same SSID as a legitimate network, some clients may connect to it automatically, especially if the signal is stronger or the client is not configured to verify the correct AP.

This creates a serious risk because the attacker-controlled AP can place itself between the client and the internet. From there, the attacker may monitor traffic, redirect users to fake login pages, or perform man-in-the-middle attacks.

Exploitation

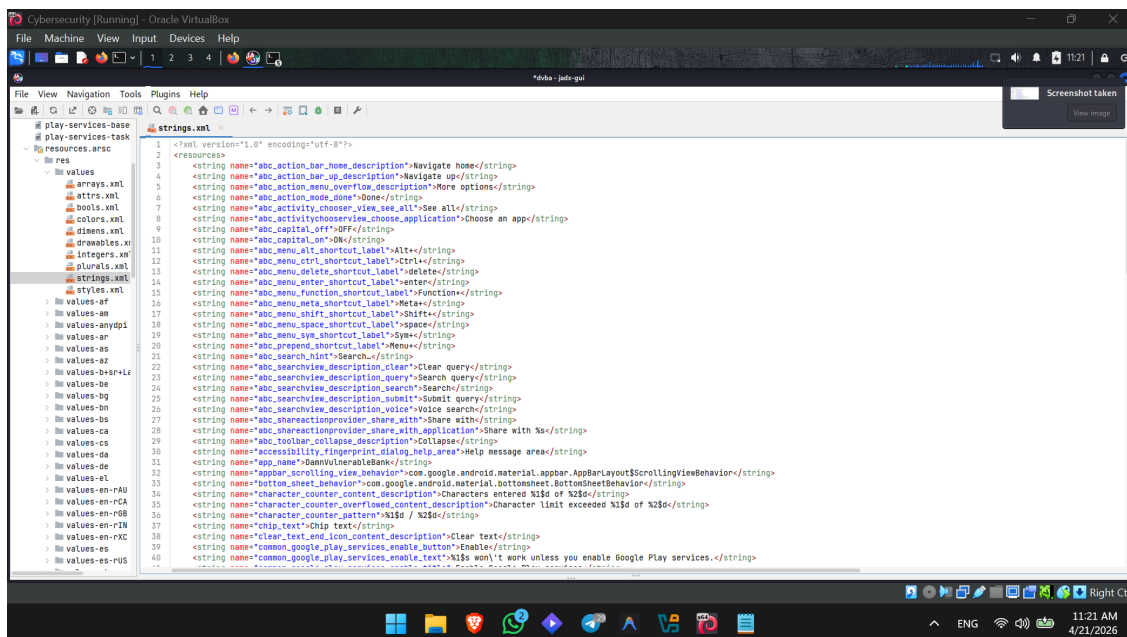
An attacker can create a rogue AP using the same SSID as the trusted network, for example [CorpLab](#). If a victim device connects to the rogue AP, the attacker may capture traffic, steal credentials, redirect DNS requests, or launch further attacks against the connected client.

In this lab, the rogue AP was successfully detected by Kismet because it had the same SSID but a different BSSID. The client was protected by forcing it to connect only to the known legitimate BSSID.

Protection / Mitigation

Use wireless monitoring tools such as Kismet to detect duplicate SSIDs, unknown BSSIDs, and suspicious AP behavior. Configure clients to connect only to trusted BSSIDs when possible, especially in controlled environments. Use WPA2/WPA3-Enterprise with certificate validation for stronger AP authentication. Users should avoid automatically connecting to networks only by SSID, and administrators should regularly monitor the wireless environment for rogue APs.

Exercise 06:



The screenshot shows the [DVBA.apk](#) application opened in [jadx](#). The static analysis focused on the decompiled resources and Java source code, especially files such as [res/values/strings.xml](#) and login/session-related classes. The goal was to identify sensitive information exposed inside the APK, such as hardcoded backend configuration, insecure local storage, and weak client-side security logic.

Extracted Information

Artifact Type	File / Location	Extracted Value	Explanation
Google API Key	res/values/strings.xml	[API KEY REDACTED]	Hardcoded cloud/API key stored in plaintext.
Firebase Database URL	res/values/strings.xml	https://damn-vulnerable-bank.firebaseio.com	Backend database endpoint exposed inside the APK.
Cloud Storage Bucket	res/values/strings.xml	damn-vulnerable-bank.appspot.com	Cloud storage bucket name exposed to anyone who decompiles the app.
Firebase Project ID	res/values/strings.xml	damn-vulnerable-bank	Identifies the Firebase project used by the application.
OAuth Web Client ID	res/values/strings.xml	932398433474-59j4a17sqbf1r9m1f2eqq...	OAuth client identifier exposed in the application resources.
JWT Storage	BankLogin.java/jwt.xml	Session token saved in SharedPreferences	Post-login token stored locally in an unencrypted XML file.

Client-side encryption logic	Decompiled Java class c.b.a.e	Encryption logic visible in source code	The app performs cryptography on the client side, so the logic can be analyzed by an attacker.
------------------------------	---	---	--

Explanation of the Extracted Data

Using [jadx](#), the APK was decompiled and its internal resources became readable. The analysis revealed that [DVBA.apk](#) contains backend configuration values directly inside the application resources. These values include a Google API key, Firebase database URL, Cloud Storage bucket, Firebase project ID, and OAuth client ID.

The analysis also showed that sensitive session data such as JWT tokens may be stored locally using standard Android storage mechanisms like SharedPreferences. Since the APK can be decompiled, any client-side security or encryption logic can also be inspected and reproduced by an attacker.

Analysis and Vulnerability

The main vulnerabilities are **hardcoded backend information**, **insecure local storage**, and **exposed client-side logic**.

Hardcoded values inside an APK are not secret because Android applications can be reverse engineered. If an attacker obtains the APK, they can use [jadx](#) to inspect resources and extract backend endpoints, API keys, and project identifiers. If the Firebase or backend access rules are weak, the attacker may use these values to access or abuse backend services.

Insecure local storage is also dangerous because session tokens saved in plaintext can be stolen from a rooted device, backup files, or malware with sufficient access. Exposed client-side encryption logic is weak because the attacker can analyze the decompiled code and understand how sensitive operations are performed.

Exploitation

An attacker can decompile [DVBA.apk](#) with [jadx](#) and extract sensitive configuration values from the resources. The exposed Firebase URL could be tested for weak access rules, and the API key could be abused if it is not restricted.

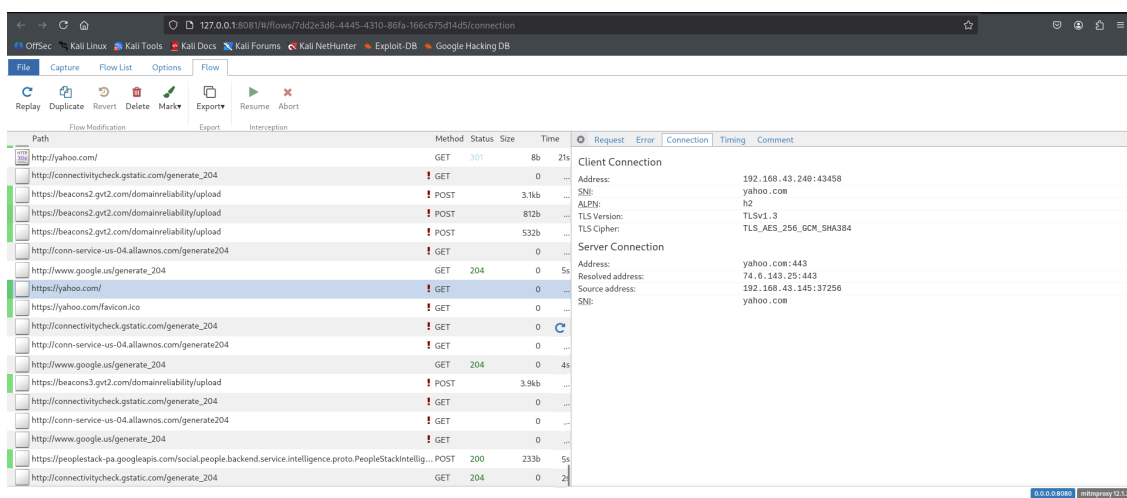
If the attacker gains access to the device storage, they may extract locally stored session data such as JWT tokens and reuse them to bypass authentication. Also, if encryption or validation logic is implemented only inside the app, the attacker can reverse engineer it and reproduce the same behavior outside the official application.

Protection / Mitigation

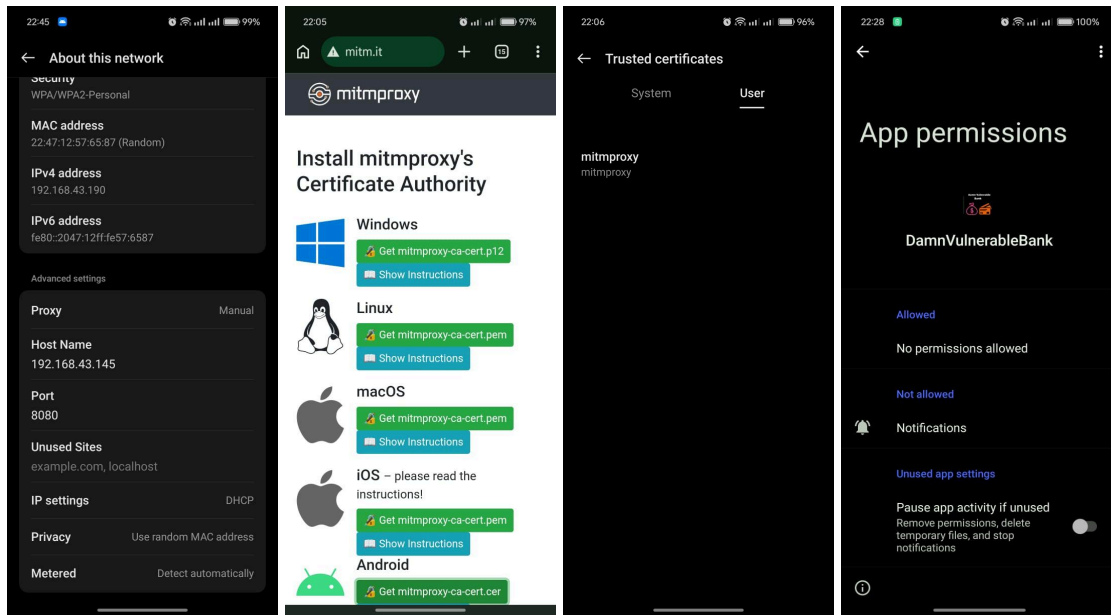
Do not store secrets directly inside the APK. Backend access should be protected by server-side authentication and authorization, not by hiding values in the mobile app. API keys should be restricted in the Google Cloud Console using the package name, SHA-1 signing certificate, and allowed APIs.

Sensitive local data such as JWT tokens should be stored using [EncryptedSharedPreferences](#) and the Android Keystore system. Firebase Security Rules must enforce strict user authentication and authorization. Critical validation and cryptographic operations should be handled on the backend whenever possible, because client-side code can always be reverse engineered.

Exercise 07:



Path	Method	Status	Size	Time	Request	Error	Connection	Timing	Comment
http://yahoo.com/	GET	200	8b	21s			Client Connection		
http://connectivitycheck.gstatic.com/generate_204	GET	204	0	...			Address: 192.168.43.240:43458		
https://beacons2.gvt2.com/domainreliability/upload	POST	200	3.1kb	...			SNI: yahoo.com		
https://beacons2.gvt2.com/domainreliability/upload	POST	200	812b	...			ALPN: h2		
https://beacons2.gvt2.com/domainreliability/upload	POST	200	532b	...			TLS Version: TLSv1.3		
https://beacons2.gvt2.com/domainreliability/upload	POST	200	532b	...			TLS Cipher: TLS_AES_256_GCM_SHA384		
http://conn-service-us-04.allwms.com/generate204	GET	204	0	...			Server Connection		
http://www.google.us/generate_204	GET	204	0	5s			Address: yahoo.com:443		
https://yahoo.com/	GET	200	0	...			Resolved address: 74.8.143.25:443		
https://yahoo.com/favicon.ico	GET	200	0	...			Source address: 192.168.43.145:37256		
http://connectivitycheck.gstatic.com/generate_204	GET	204	0	...			SNI: yahoo.com		
http://connectivitycheck.gstatic.com/generate_204	GET	204	0	...					
http://conn-service-us-04.allwms.com/generate204	GET	204	0	...					
http://www.google.us/generate_204	GET	204	0	4s					
https://beacons3.gvt2.com/domainreliability/upload	POST	200	3.9kb	...					
http://connectivitycheck.gstatic.com/generate_204	GET	204	0	...					
http://conn-service-us-04.allwms.com/generate204	GET	204	0	...					
http://conn-service-us-04.allwms.com/generate204	GET	204	0	...					
https://www.google.us/generate_204	GET	204	0	...					
https://peoplestack-pa.googleapis.com/social.people.backend.service.intelligence.proto.PeopleStackIntelli...	POST	200	233b	5s					
http://connectivitycheck.gstatic.com/generate_204	GET	204	0	21s					



Implementation Process

The test was performed using a Kali virtual machine and a physical Android phone instead of an Android emulator. First, the Kali VM IP address was identified and used as the proxy host on the Android phone.

Next, Mitmproxy was started in web mode using:

[`mitmweb --listen-host 0.0.0.0 --listen-port 8080`](#)

The option [`--listen-host 0.0.0.0`](#) allowed the proxy to accept connections from other devices on the network, and port [`8080`](#) was used as the proxy port.

After that, the Android phone was configured to use the Kali VM as a manual Wi-Fi proxy. The proxy hostname was set to the Kali IP address, and the proxy port was set to [`8080`](#).

Then, the phone browser was used to generate normal web traffic. This traffic appeared successfully in [`mitmweb`](#), which confirmed that the proxy configuration was working correctly.

Finally, the DVBA application was opened on the Android phone. A username and password were entered in the login form, but the application displayed an error message. No DVBA-related traffic appeared in [`mitmweb`](#). The phone's application permission settings were also checked, and no permissions were allowed for the app except optional notifications.

Extracted Information

Field	Extracted Value	Explanation
Proxy tool	mitmweb / Mitmproxy	Used to intercept and inspect phone traffic.
Proxy machine	Kali VM	The virtual machine acted as the interception proxy.
Client device	Android phone	Physical Android phone used instead of an emulator.
Proxy host	Kali VM IP address	The phone was configured to send traffic to this IP.
Proxy port	8080	Listening port used by Mitmproxy.
Browser traffic result	Visible in mitmweb	Confirms that the phone proxy setup worked.
Target application	DVBA.apk	Android app tested for MITM interception.
DVBA traffic result	Not visible in mitmweb	The app traffic was not successfully intercepted.
App behavior	Login error / “something wrong”	The app failed after entering credentials.
App permissions	No allowed permissions except notifications	No sensitive Android permissions were enabled from phone settings.

Explanation of the Extracted Data

The successful appearance of normal browser traffic in [mitmweb](#) proves that the Kali proxy, Android proxy settings, IP address, and port were configured correctly. Therefore, the lack of DVBA traffic was not caused by a general proxy setup failure.

When the DVBA login form was tested, the app displayed an error and no related request appeared in Mitmproxy. This indicates that the application traffic was not intercepted in this setup. The reason may be that the app rejected the proxy connection, did not trust the Mitmproxy certificate, ignored the system proxy, used strict TLS validation, or failed before sending the request.

The app permission page showed that no permissions were allowed except optional notifications. This means no visible user-granted permission was responsible for the failed interception.

Analysis and Vulnerability

The expected vulnerability in this exercise is **improper certificate pinning**. If an Android app accepts a user-installed CA certificate and does not verify the server certificate properly, Mitmproxy can intercept and display sensitive HTTPS traffic.

In this test, normal browser traffic was intercepted successfully, but DVBA traffic did not appear in Mitmproxy. Therefore, this test does not prove that DVBA is vulnerable to MITM interception in the current setup. Instead, the result suggests that DVBA may reject the Mitmproxy certificate, ignore the proxy configuration, use strict TLS validation, or fail before the network request is sent.

Exploitation

If a vulnerable Android application does not enforce proper certificate validation or certificate pinning, an attacker on the same network could redirect the victim's traffic through a malicious proxy. If the attacker's certificate is trusted, they could inspect sensitive data such as usernames, passwords, tokens, and API requests.

In this test, exploitation was not successfully demonstrated against DVBA because no DVBA-related traffic appeared in [mitmweb](#). Only normal browser traffic was successfully intercepted.

Protection / Mitigation

Applications should enforce HTTPS and correctly validate server certificates. Sensitive mobile applications should implement certificate pinning so they only trust the legitimate backend certificate or public key. They should not blindly trust user-installed CA certificates in production environments.

Sensitive data such as usernames, passwords, and session tokens should never be sent over plain HTTP. The app should also provide clear error handling when secure connections fail, instead of silently failing or showing a vague error message.

Exercise 08:

```
(kali@kali)-[~]
$ apktool d sieve.apk -o SieveApp
I: Using Apktool 2.7.0-dirty on sieve.apk
I: Loading resource table ...
I: Decoding AndroidManifest.xml with resources ...
I: Loading resource table from file: /home/kali/.local/share/apktool/framework/1.apk
I: Regular manifest package ...
I: Decoding file-resources ...
I: Decoding values */* XMLs ...
I: Baksmaling classes.dex ...
I: Copying assets and libs ...
I: Copying unknown files ...
I: Copying original files ...

(kali@kali)-[~]
$ cd SieveApp
cat AndroidManifest.xml
<?xml version="1.0" encoding="utf-8" standalone="no"?><manifest xmlns:android="http://schemas.android.com/apk/res/android" package="com.mwr.example.sieve">
  <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />
  <uses-permission android:name="android.permission.INTERNET" />
  <permission android:label="Allows reading of the Key in Sieve" android:name="com.mwr.example.sieve.READ_KEYS" android:protectionLevel="dangerous"/>
  <permission android:label="Allows editing of the Key in Sieve" android:name="com.mwr.example.sieve.WRITE_KEYS" android:protectionLevel="dangerous"/>
  <application android:allowBackup="true" android:debuggable="true" android:icon="@drawable/ic_launcher" android:label="@string/app_name" android:theme="@s
tyle/AppTheme">
    <activity android:clearTaskOnLaunch="true" android:excludeFromRecents="true" android:exported="true" android:finishOnTaskLaunch="true" android:label=
"@string/title_activity_file_select" android:name=".FileSelectActivity" />
    <activity android:excludeFromRecents="true" android:label="@string/app_name" android:launchMode="singleTask" android:name=".MainLoginActivity" androi
d:windowSoftInputMode="adjustResize|stateVisible">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>
  </application>
</manifest>
```

The screenshots show the decoded [AndroidManifest.xml](#) file of [Sieve.apk](#) after using [apktool](#). The analysis focused on requested permissions, dangerous permissions, insecure application flags, and exported components such as activities, services, and content providers.

Extracted Information

Vulnerability Type	Target Artifact / Manifest Entry	Extracted Finding	Explanation
Overprivilege	<uses-permission android:name="android.permission.INTERNET"/>	Internet permission requested	The app is a local password manager, so internet access may be unnecessary and increases the risk of data exfiltration.

Weak custom permissions	<u>READ_KEYS / WRITE_KEYS</u> with <u>protectionLevel="dangerous"</u>	Custom permissions are not signature-level	A malicious app may request these permissions and interact with sensitive app data if granted.
Insecure backup flag	<u>android:allowBackup="true"</u>	Backup is enabled	App data may be extracted through backup mechanisms, especially with physical or debugging access.
Debuggable application	<u>android:debuggable="true"</u>	Debugging is enabled	Attackers can attach a debugger and inspect app behavior or sensitive data at runtime.
Exported activity	<u><activity android:name=".PWList"</u> <u>android:exported="true"/></u>	Password list activity is exported	Other apps may launch this activity directly and bypass normal login flow.
Exported provider	<u><provider</u> <u>android:name=".FileBackupProvider"</u> <u>android:exported="true"/></u>	Content provider is globally exposed	Other apps may query the provider and attempt to access sensitive files.

Explanation of the Extracted Data

Using [apktool](#), the APK was decoded and the [AndroidManifest.xml](#) file became readable. The manifest revealed several security problems: the app requests

permissions that may not match its real function, allows backup, enables debugging, and exposes internal components.

The most critical findings are the exported [PWList](#) activity and the exported [FileBackupProvider](#). Since Sieve is a password manager, these components are sensitive. If they are exported without strong permission checks, another application on the same device may interact with them through Android IPC.

Analysis and Vulnerability

The main vulnerability is **insecure Android component exposure**. Android apps can communicate through intents and content providers. If sensitive activities or providers are exported, malicious apps may access them without passing through the intended authentication process.

Another vulnerability is **insecure application configuration**. The flags [allowBackup="true"](#) and [debuggable="true"](#) are dangerous for a password manager because they make data extraction and runtime inspection easier.

The custom permissions are also weak because dangerous-level permissions can be granted to other apps. For sensitive operations such as reading or writing keys, signature-level permissions would be safer.

Exploitation

An attacker could install a malicious app on the same phone and send an intent to the exported [PWList](#) activity. If the activity does not enforce authentication internally, the attacker may bypass the login screen and access the password list.

The exported [FileBackupProvider](#) could also be abused by another app to query or extract sensitive files. If backup is enabled, an attacker with physical or debugging access may attempt to extract app data. If debugging is enabled, the attacker may attach a debugger and inspect decrypted passwords or sensitive values in memory.

Protection / Mitigation

Set internal activities, services, receivers, and providers to [android:exported="false"](#) unless they must be accessed by other apps. Sensitive components should enforce authentication and permission checks internally.

For production builds, disable backup and debugging by setting [android:allowBackup="false"](#) and [android:debuggable="false"](#). Custom permissions protecting sensitive data should use [protectionLevel="signature"](#) so only apps signed

with the same developer certificate can access them. The app should also follow the principle of least privilege and request only permissions required for its real features.

Exercise 09:

```
(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ touch secure_config.conf

(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ sudo chattr +i secure_config.conf
[sudo] password for kali:

(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ lsattr secure_config.conf
---i-----e----- secure_config.conf

(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ sudo rm secure_config.conf
rm: cannot remove 'secure_config.conf': Operation not permitted

(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ sudo chattr -i secure_config.conf
```

```
(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ touch audit_log.txt

(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ sudo chattr +a audit_log.txt

(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ echo "User logged in" >> audit_log.txt

(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ echo "Erased" > audit_log.txt
zsh: operation not permitted: audit_log.txt
```

The screenshot shows the use of Linux file attributes with the [chattr](#) and [lsattr](#) commands. In this exercise, the goal was to demonstrate how special file attributes can protect important files from modification, deletion, or unauthorized changes.

The exercise sheet asks to practice using the Linux [chattr](#) command to protect and control file behavior.

Extracted Information

Command / Attribute	Meaning	Security Effect
chattr +i file	Adds the immutable attribute	Prevents file modification, deletion, renaming, or overwriting.
chattr -i file	Removes the immutable attribute	Allows the file to be modified again.
lsattr file	Lists file attributes	Used to verify whether special attributes are active.
chattr +a file	Adds append-only mode	Allows data to be added, but prevents deleting or overwriting existing content.
chattr -a file	Removes append-only mode	Restores normal write behavior.

Explanation of the Extracted Data

The [chattr](#) command controls special Linux file attributes that go beyond normal permissions such as [chmod](#). The immutable attribute [+i](#) makes a file resistant to modification even if the user has normal write permissions. The append-only attribute [+a](#) is useful for logs because new entries can be added, but previous content cannot be overwritten or deleted.

This is useful for protecting critical files, configuration files, and logs from accidental or malicious changes.

Analysis and Vulnerability

The vulnerability demonstrated here is **unrestricted file modification after privilege escalation**.

Normal Linux permissions are useful, but if an attacker gains root access, they can usually modify or delete important files. Your draft identifies examples such as [/etc/shadow](#), [/etc/crontab](#), [/var/log/auth.log](#), [/var/log/syslog](#), and web directories as sensitive targets that could be modified without stronger file protection.

Without extra protection, an attacker could edit system files to create persistence, wipe logs to hide activity, or modify web files for defacement.

Exploitation

If an attacker gains high privileges, they may overwrite files such as [/etc/crontab](#) to create persistence, modify [/etc/shadow](#) to affect user accounts, or erase [/var/log/auth.log](#) to hide login evidence. They may also modify web directories to deface a website or encrypt shared folders during a ransomware attack.

Using [chattr +i](#) or [chattr +a](#) makes these actions harder because protected files cannot be freely changed unless the attribute is removed first.

Protection / Mitigation

Use [chattr +i](#) to protect critical configuration files that should not change frequently. Use [chattr +a](#) for log files where new entries should be added but old entries should not be overwritten. Regularly verify attributes with [lsattr](#).

This should not be the only defense. It should be combined with strong access control, least privilege, backups, log monitoring, intrusion detection, and restricting root access.

Exercise 10:

```
(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ cp /usr/bin/tcpdump ~/my_tcpdump

(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ ./my_tcpdump -i eth0
zsh: no such file or directory: ./my_tcpdump

(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ ~/my_tcpdump -i eth0
my_tcpdump: eth0: You don't have permission to perform this capture on that device
(socket: Operation not permitted)

(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ sudo setcap 'cap_net_raw,cap_net_admin+eip' ~/my_tcpdump

(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ getcap ~/my_tcpdump
/home/kali/my_tcpdump cap_net_admin,cap_net_raw=eip

(kali㉿kali)-[~/Desktop/ModdedCalculator/original]
$ ~/my_tcpdump -i eth0
my_tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
10:31:36.891722 IP 10.0.2.15.58395 > 10.0.2.3.domain: 64796+ A? telemetry.elastic.co. (38)
```

The screenshot shows the process of checking the current capabilities of the selected binary, assigning a limited capability to it, verifying that the capability was added, and testing that the binary works without full root privileges.

The goal of this exercise is to demonstrate how Linux capabilities can give a program only the privilege it needs, instead of giving it complete root access.

Implementation Process

First, the current capabilities of the selected binary were checked. In this exercise, the [ping](#) binary was used because it needs permission to create raw network packets.

After that, the [CAP_NET_RAW](#) capability was assigned to the binary. This capability allows the program to create raw network sockets, which are required for sending ICMP packets.

Then, the capabilities were checked again to verify that the assignment was successful. Finally, the binary was tested as a normal user to confirm that it could still perform its function without being executed with full administrator privileges.

Extracted Information

The main tools used in this exercise were [getcap](#) and [setcap](#). The [getcap](#) tool was used to display the capabilities assigned to the selected binary, while [setcap](#) was used to assign a specific capability.

The target binary was [ping](#), which requires the ability to create raw network packets. The assigned capability was [CAP_NET_RAW](#). This capability allows the binary to send ICMP packets without needing full root privileges.

The extracted result shows that Linux capabilities can replace the older all-or-nothing privilege model. Instead of running the whole program as root, only the required network privilege was granted.

Explanation of the Extracted Data

The [CAP_NET_RAW](#) capability gives the [ping](#) program permission to create raw network packets. This is necessary because [ping](#) sends ICMP packets, which normally require elevated privileges.

Instead of making the program run as root, Linux capabilities allow only this specific privilege to be granted. This reduces risk because the program does not receive complete administrator access. It only receives the network-related capability required for its function.

Analysis and Vulnerability

The vulnerability demonstrated in this exercise is **all-or-nothing privilege escalation caused by SUID binaries or excessive administrator privileges**.

Traditionally, some binaries were given full root privileges so normal users could perform specific tasks. This is dangerous because if a vulnerable binary runs with full root privileges, an attacker may exploit it to gain complete control of the system.

Using capabilities is safer because privileges are separated into smaller units. Instead of giving the whole binary full root power, the system grants only the exact privilege required.

Exploitation

If a binary has full root privileges and contains a vulnerability, an attacker may exploit it to execute commands as root. This could lead to full system compromise, privilege escalation, data theft, persistence, or modification of sensitive files.

With Linux capabilities, the impact is reduced. If the [ping](#) binary only has [CAP_NET_RAW](#), then even if something goes wrong, the binary is not automatically granted every root privilege. The attacker's possible actions are more limited compared to exploiting a full SUID-root program.

Protection / Mitigation

Use Linux capabilities instead of SUID whenever possible. Assign only the exact capability needed by the program and avoid giving broad privileges to complex or untrusted binaries.

Administrators should regularly audit binaries with special privileges, remove unnecessary capabilities, and avoid relying on full root execution for simple tasks. This follows the principle of least privilege and reduces the risk of complete system compromise if a privileged binary is exploited.

Conclusion

This lab demonstrated several important security concepts across wireless networks, Android applications, and Linux systems. Through Wi-Fi analysis, we observed that management frames expose useful metadata such as SSID, BSSID, channel, and security configuration. Although this information is part of normal wireless operation, it can help attackers perform reconnaissance and prepare targeted attacks.

The WPA/WPA2 exercises showed that capturing a handshake does not reveal the password directly, but it allows offline password guessing. The dictionary attack results proved that weak passphrases such as common words or simple numbers can be recovered quickly. This highlights the importance of using long, random Wi-Fi passwords and disabling insecure features such as WPS.

The mobile application exercises showed that Android applications can expose sensitive information through hardcoded keys, insecure local storage, exported components, weak permissions, or improper TLS handling. Reverse engineering and manifest analysis demonstrated that APK files should not be trusted to hide secrets, because attackers can inspect their resources and code.

The Linux exercises showed how system hardening can reduce the impact of attacks. File attributes such as immutable and append-only modes can protect critical files and logs, while Linux capabilities allow programs to receive only the specific privileges they need instead of full root access.

Overall, the lab shows that security depends not only on encryption or tools, but also on correct configuration, strong authentication, least privilege, secure coding, monitoring, and defense in depth.